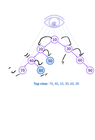
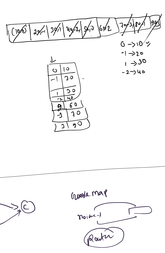
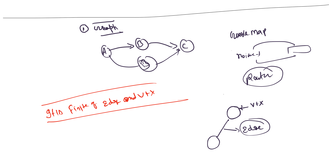
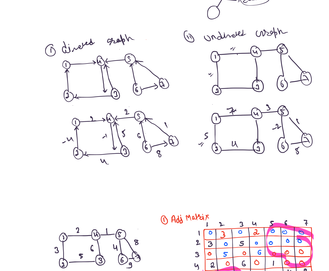
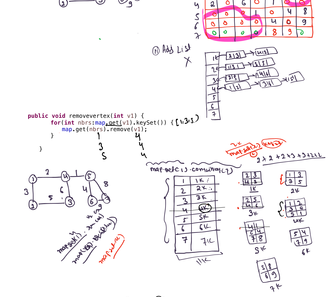
Lec-37

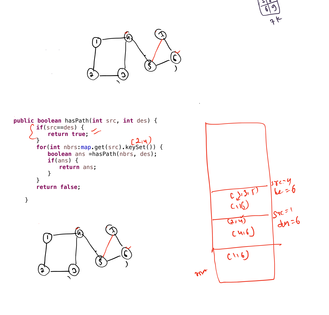


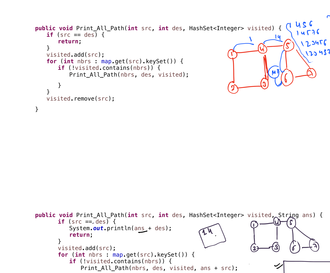




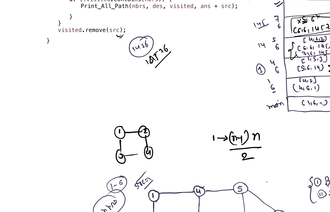


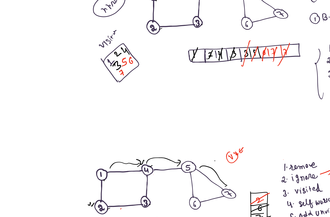




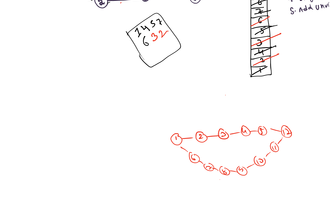


```
public void removeVertex(int v1) {
    for(int nbrs:map.get(v1).keySet()) {
        map.get(nbrs).remove(v1);
    }
}
```

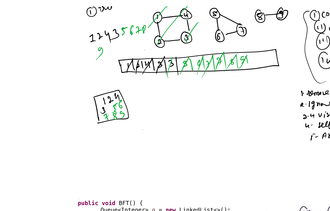




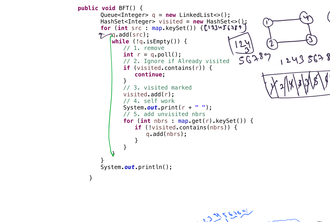
```
public boolean hasPath(int src, int des) {
    if(srcvsdes) {
        return true;
    }
    for(int nbrs:map.get(src).keySet()) {
        boolean ans =hasPath(nbrs, des);
        if(ans) {
            return ans;
        }
    }
    return false;
}
```

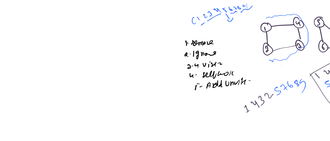


```
public void Print_All_Path(int src, int des, HashSet<Integer> visited) {
    if (src == des) {
        return;
    }
    visited.add(src);
    for (int nbrs : map.get(src).keySet()) {
        if (!visited.contains(nbrs)) {
            Print_All_Path(nbrs, des, visited);
        }
    }
    visited.remove(src);
}
```



```
public void Print_All_Path(int src, int des, HashSet<Integer> visited, String ans) {
        if (src == des) {
            System.out.println(ans + des);
            return;
        }
        visited.add(src);
        for (int nbrs : map.get(src).keySet()) {
            if (!visited.contains(nbrs)) {
                Print_All_Path(nbrs, des, visited, ans + src);
            }
        }
        visited.remove(src);
    }
```






```
public void BFT() {
    Queue<Integer> q = new LinkedList<>();
    HashSet<Integer> visited = new HashSet<>();
    for (int src : map.keySet()) {
        q.add(src);
        while (!q.isEmpty()) {
            // 1. remove
            int r = q.poll();
            // 2. Ignore if already visited
            if (visited.contains(r)) {
                continue;
            }
            // 3. visited marked
            visited.add(r);
            // 4. work
            System.out.print(r + " ");
            // 5. add unvisited nbrs
            for (int nbrs : map.get(r).keySet()) {
                if (!visited.contains(nbrs)) {
                    q.add(nbrs);
                }
            }
        }
        System.out.println();
    }
```